

# Scripts and images\*

## Distributed Data Processing Environments

### Lab Guide 1

This session aims at automating the environment built by hand in Lab 0, first with a shell script and then with an image. Set up your coding agent as described in the AI protocol before starting. Keep your files on your own machine: the script runs inside the Lab 0 container, and the image is built outside it. Nothing in this session consumes credits.

**Steps** First, inside the Lab 0 container, replace the commands you typed there with a script:

1. Create a file `setup.sh` holding the `apt` and `pip` commands of Lab 0, in order.
2. Make it executable with `chmod +x setup.sh` and run it with `./setup.sh`
3. Add a first line `#!/bin/bash` and explain what it does.
4. Accept the name of a package as a parameter, available in the script as `$1`.
5. Run the script a second time. Which commands did useful work, and which did not?

Now, on your own machine, build an image that already contains the environment:

1. Create a file `Dockerfile` containing:

```
FROM ubuntu:24.04

RUN apt-get update && \
    apt-get install -y python3-pip python3-venv

WORKDIR /app
COPY download.py .

RUN python3 -m venv venv && \
    ./venv/bin/pip install requests
```

2. Build it with: `docker build -t adpd-lab1 .`
3. Note how long the build took. Build it a second time, without changing anything, and compare.
4. Run the program from the image, writing into your folder:  
`docker run --rm -v ${PWD}:/out adpd-lab1 ./venv/bin/python3 download.py`
5. Start two containers from this one image, create a file in each, and check whether the other one sees it.
6. Explain the difference between the image and a container.

Finally, compare the two ways of arriving at the same environment:

1. State which of `setup.sh` and `Dockerfile` runs once, and which runs every time.

---

\*Tutor mode throughout. Reviewer mode in the last part. See AI protocol.

2. Ask the agent to criticize your `Dockerfile`. Rule on each criticism, running the build again where it helps.
3. Ask the agent which files it can read. Then test its answer: put a file outside the folder and ask it for the contents.

At the end of the session:

1. Remove your containers with `docker rm`, and confirm with `docker ps -a`.
2. Confirm with `docker images` that the image is still there, and explain why.

**Learning Outcomes** Automate a sequence of commands with a shell script. Build an image holding a program and the software it needs. Distinguish an image from a container. Identify what an agent can read from your machine.